



Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Estado de México

M1. Actividad

Luis David Pozos Tamez A01800657

Adrián Proaño Bernal A01752615

Modelación de Sistemas Multiagentes con Gráficas Computacionales (Grupo 302)

Profesor:

Jorge Adolfo Ramírez Uresti

Introducción:

Para esta actividad, se nos pidió utilizar el conocimiento adquirido en las primeras dos semanas de clase, para simular robots de limpieza reactivos. Se nos pidió que le pasáramos los siguientes datos a la simulación:

- Habitación de $M \times N$ espacios.
- Número de agentes de la simulación.
- Porcentaje de celdas inicialmente sucias.
- Tiempo máximo de simulación.

Los agentes iniciaran en la celda $[1,1]$, y cada paso de la simulación, los agentes primero revisarán si la celda está sucia. Si la celda está sucia, limpiar la celda, En el caso de que la celda esté limpia, el agente escoge una dirección aleatoria e intentara moverse. El agente no se podrá mover si ya hay otro agente en la dirección que escogió.

Explicación de código:

Se nos pidió que ejecutáramos esta simulación con varios valores, con el propósito de analizar el impacto que tiene la cantidad de agentes sobre el tiempo de ejecución, y la cantidad de movimientos. A continuación, mostraremos nuestros bloques de código, explicando que fue lo que hicimos en cada uno:

```
Python
# type: ignore
# Has multi-dimensional arrays and matrices.
# Has a large collection of mathematical functions to operate on these arrays.
import numpy as np

# Data manipulation and analysis.
import pandas as pd
```

```

# Data visualization tools.
import seaborn as sns

# Agent functionality
import mesa

from mesa.discrete_space import Cell, CellAgent, OrthogonalMooreGrid

import random

import time

from matplotlib.colors import ListedColormap

class DirtyCell(Cell):
    def __init__(self, coordinate, capacity, random, dirty_percentage):
        super().__init__(coordinate, capacity, random)
        self.isClean = True if random.randint(1, 100) > dirty_percentage else
False

```

Primero, tenemos que asegurarnos que las celdas tengan la información de su estado, ya sea que estén limpias o no. Creamos una clase “DirtyCell” que hereda de la clase padre “Cell”. En el constructor, la inicializamos como una celda normal, pero también pasamos el porcentaje que de celdas que queremos sucias y agregamos un valor booleano que nos dirá si está limpia o no. No nos preocupamos por asignarlo manualmente ya que al inicializarla, se obtiene un número entre 1 y 100, y si ese número es menor al número del porcentaje, la celda será clasificada como “sucia”.

```

Python
class CleaningAgent(CellAgent):

    def __init__(self, model, cell):

```

```

    super().__init__(model)
    self.cell = cell
    self.move_counter = 0

    def move(self):
        neighbors = list(self.cell.neighborhood.cells)

        empty_neighbors = [
            cell for cell in neighbors
            if len(cell.agents) == 0
        ]

        if empty_neighbors:
            self.cell = random.choice(empty_neighbors)
            self.move_counter += 1

    def clean_cell(self):
        self.cell.isClean = True

```

Aquí creamos nuestro agente que se encargará de moverse de celda a celda y limpiarlas. Lo inicializamos pasando el modelo al cual el agente pertenece, la celda en la cual se encuentra y la cantidad de movimientos que ha realizado el agente. El agente tiene una función “move”, la cual primero, obtiene una lista de las celdas vecinas a sí mismo, después, guarda en una lista las celdas que no tienen agentes, finalmente, revisa si existe al menos una celda vacía, en caso de que si, selecciona una de ellas al azar, y se mueve a ella. También nos aseguramos que se le agregue al contador de sus movimientos totales.

Su segunda función simplemente asigna el estado “isClean” de la celda en la cual se encuentra a “True”.

Python

"""

Modelo de limpieza con agentes que se mueven y limpian celdas sucias.
"""

```
class CleaningModel(mesa.Model):

    def __init__(self, n, width, height, dirty_percentage, seed=None):
        super().__init__(seed=seed)
        self.num_agents = n
        self.dirty_percentage = dirty_percentage

        class DirtyCellWithPercentage(DirtyCell):
            def __init__(self, coordinate, capacity, random):
                super().__init__(coordinate, capacity, random,
dirty_percentage)

        self.grid = OrthogonalMooreGrid(
            (width, height),
            torus=True,
            capacity=100,
            random=self.random,
            cell_klass=DirtyCellWithPercentage
        )

        initial_cell = self.grid[(1,1)]

        agents = CleaningAgent.create_agents(
            self,
            self.num_agents,
            [initial_cell] * self.num_agents
        )

    def step(self):
        self.agents.do("move")
        self.agents.do("clean_cell")
```

Aquí creamos el modelo completo de limpieza, con los agentes y el ambiente en el cual se mueven e interactúan entre sí. El constructor requiere todos los elementos necesarios para ejecutar la simulación, incluyendo la probabilidad de que una celda sea instanciada como

“sucia”. Debido a limitaciones de la librería Mesa, fue necesario definir una subclase para que DirtyCell pudiera recibir dicho valor sin romper el funcionamiento de “Cell”.

Después de eso, el constructor inicializa un grid que define la capacidad de las celdas y les asigna la propiedad de isClean. También se asegura de que todos los agentes sean creados en la posición [1, 1].

Por último, se define lo que queramos que haga nuestra simulación por cada “step” que tome. Cada agente presente intentará moverse a una celda adyacente que se encuentre libre, comprobará si está sucia, y la limpiará si ese es el caso. Toda la lógica necesaria para dicho funcionamiento está encapsulada en “move” y “clean_cell”.

Una vez que definimos nuestro modelo, simplemente falta correrlo. En el siguiente bloque de código se definen los parámetros y el modelo corre hasta que limpia todas las celdas o agote el tiempo máximo de ejecución:

```
Python
n_agents = 70
width = 20
height = 20
dirty_percentage = 60
model = CleaningModel(n=n_agents, width=width, height=height,
dirty_percentage=dirty_percentage)

"""
Aquí se corre el modelo hasta que todas las celdas estén limpias o hasta que se
alcance el tiempo máximo permitido.
Al finalizar, se imprime el resultado y el número de movimientos realizados por
cada agente.
"""

max_time = 0.01 # segundos
start_time = time.time()
step_count = 0
```

```

while (time.time() - start_time) < max_time:
    model.step()
    step_count += 1

    # Verificar si todas las celdas están limpias
    all_clean = all(cell.isClean for cell in model.grid.all_cells)
    if all_clean:
        elapsed = time.time() - start_time
        print(f"✓ Todas las celdas limpias en {step_count} pasos
({elapsed:.2f}s)")
        # Imprimir número de movimientos por agente
        for i, agent in enumerate(model.agents):
            print(f" - Agente {i+1}: {agent.move_counter} movimientos")
            break
    else:
        dirty_cells = sum(1 for cell in model.grid.all_cells if not cell.isClean)
        percentage = (dirty_cells / (width * height)) * 100
        print(f"Tiempo máximo alcanzado ({max_time}s). Quedan {dirty_cells} celdas
sucias (un {percentage:.2f} % del total)")
        # Imprimir número de movimientos por agente
        for i, agent in enumerate(model.agents):
            print(f" - Agente {i+1}: {agent.move_counter} movimientos")

#Output: Tiempo máximo alcanzado (0.01s). Quedan 105 celdas sucias (un 26.25 %
del total)
#- Agente 1: 19 movimientos
#- Agente 2: 19 movimientos
#- Agente 3: 19 movimientos
#- Agente 4: 19 movimientos
#- Agente 5: 19 movimientos
#- Agente 6: 19 movimientos
#- Agente 7: 19 movimientos
#- Agente 8: 19 movimientos
#- Agente 9: 18 movimientos
#- Agente 10: 18 movimientos
#- Agente 11: 18 movimientos
#- Agente 12: 17 movimientos
#- Agente 13: 17 movimientos
#- Agente 14: 17 movimientos
#- Agente 15: 17 movimientos
#- Agente 16: 17 movimientos
#- Agente 17: 17 movimientos
#- Agente 18: 17 movimientos

```

```
#- Agente 19: 16 movimientos
#- Agente 20: 16 movimientos
#- Agente 21: 16 movimientos
#- Agente 22: 16 movimientos
#- Agente 23: 15 movimientos
#- Agente 24: 15 movimientos
#...
#- Agente 67: 7 movimientos
#- Agente 68: 7 movimientos
#- Agente 69: 7 movimientos
#- Agente 70: 7 movimientos
```

Conclusiones

Corrimos esta misma simulación con diferentes números de agentes, y pudimos observar que normalmente, cuando manejamos una cantidad de agentes muy baja, la cantidad de movimientos y el tiempo de ejecución incrementan bastante a diferencia de si tuvieramos muchos agentes, ya que como los agentes escogen de forma aleatoria a qué celda moverse, mientras más celdas limpian, se vuelve más complicado que terminen de limpiar el resto de las celdas, ya que es muy común que vayan a celdas ya limpias varias veces. Al incrementar el número de agentes, hay más posibilidades de que hagan menos movimientos en total, ya que no solo terminan de limpiar las celdas más rápido, pero además, no puede hacer movimientos si hay otro agente en la celda a la cual quieren ir.